

ĐẠI HỌC BÁCH KHOA HÀ NỘI  
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



**BÁO CÁO PROJECT 1**

**ĐỀ TÀI:**

**XÂY DỰNG HỆ THỐNG HỖ TRỢ SOẠN THẢO MÔ  
TẢ CÔNG VIỆC VÀ SINH CÂU HỎI PHỎNG VẤN  
ỨNG DỤNG RAG**

|                              |                                   |
|------------------------------|-----------------------------------|
| <b>Sinh viên thực hiện:</b>  | HỌ VÀ TÊN SINH VIÊN               |
| <b>Mã số sinh viên:</b>      | MÃ SỐ SINH VIÊN                   |
| <b>Lớp / hệ đào tạo:</b>     | LỚP / CHƯƠNG TRÌNH ĐÀO TẠO        |
| <b>Giảng viên hướng dẫn:</b> | HỌC HÀM, HỌC VỊ VÀ TÊN GIẢNG VIÊN |
| <b>Học kỳ:</b>               | Học kỳ 2025.2                     |

Hà Nội, tháng 6 năm 2026

## **LỜI CAM KẾT**

Tôi, HỌ VÀ TÊN SINH VIÊN, mã số sinh viên MÃ SỐ SINH VIÊN, cam kết báo cáo PROJECT 1 với đề tài “XÂY DỰNG HỆ THỐNG HỖ TRỢ SOẠN THẢO MÔ TẢ CÔNG VIỆC VÀ SINH CÂU HỎI PHÒNG VẤN ỨNG DỤNG RAG” là kết quả học tập, nghiên cứu và phát triển phần mềm của cá nhân tôi dưới sự hướng dẫn của HỌC HÀM, HỌC VỊ VÀ TÊN GIẢNG VIÊN. Các số liệu thực nghiệm trong báo cáo được thu thập từ hệ thống đã triển khai tại thời điểm thực hiện; các tài liệu, thuật toán và công nghệ được tham khảo đều được chỉ dẫn trong danh mục tài liệu tham khảo. Tôi không sao chép nội dung của công trình khác dưới danh nghĩa kết quả của bản thân và chịu trách nhiệm về tính trung thực của báo cáo.

Hà Nội, ngày ... tháng ... năm 2026

**Sinh viên thực hiện**

HỌ VÀ TÊN SINH VIÊN

## LỜI CẢM ƠN

Tôi xin trân trọng cảm ơn HỌC HÀM, HỌC VỊ VÀ TÊN GIẢNG VIÊN đã định hướng đề tài, góp ý về phương pháp nghiên cứu và hỗ trợ tôi trong quá trình thực hiện Project 1. Tôi cũng xin cảm ơn các thầy, cô của TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG đã cung cấp nền tảng kiến thức về kỹ nghệ phần mềm, cơ sở dữ liệu, học máy và hệ thống thông tin. Cuối cùng, tôi cảm ơn gia đình và bạn bè đã động viên, tạo điều kiện để tôi hoàn thành sản phẩm và báo cáo này. Những thiếu sót còn lại trong báo cáo thuộc trách nhiệm của tác giả và là cơ sở để tiếp tục hoàn thiện hệ thống trong các giai đoạn sau.

## TÓM TẮT

Quá trình tuyển dụng đòi hỏi chuyên viên nhân sự xây dựng mô tả công việc nhất quán, xác định tiêu chí đánh giá và chuẩn bị câu hỏi phỏng vấn phù hợp với từng vị trí. Khi dữ liệu tham khảo phân tán và việc soạn thảo phụ thuộc nhiều vào kinh nghiệm cá nhân, nội dung dễ thiếu chuẩn hóa, khó truy vết nguồn và tốn thời gian cập nhật. Báo cáo trình bày SmartHire Composer, một hệ thống web hỗ trợ tạo, chỉnh sửa, quản lý phiên bản mô tả công việc và sinh câu hỏi phỏng vấn, trong đó kỹ thuật sinh tăng cường bởi truy hồi (Retrieval-Augmented Generation – RAG) được sử dụng để đưa tri thức từ các tin tuyển dụng thật vào ngữ cảnh của mô hình ngôn ngữ.

Giải pháp gồm giao diện React/Vite, dịch vụ FastAPI, PostgreSQL cho dữ liệu nghiệp vụ và tìm kiếm toàn văn, Milvus cho tìm kiếm vector, cùng mô hình Gemini cho sinh văn bản và embedding. Dữ liệu được thu thập từ API công khai của các hệ thống quản trị tuyển dụng, chuẩn hóa kèm thông tin nguồn, chia thành các đoạn có chồng lấn và ánh xạ sang vector 768 chiều. Khi truy vấn, hệ thống kết hợp truy hồi dense theo độ tương đồng cosine và truy hồi lexical bằng PostgreSQL Full-Text Search; hai danh sách được hợp nhất bằng Reciprocal Rank Fusion và lọc đa dạng để hạn chế nhiều đoạn trùng từ cùng một JD. Hệ thống hỗ trợ JWT, RBAC, lịch sử phiên bản, xuất PDF/DOCX và cơ chế dự phòng khi dịch vụ LLM không sẵn sàng.

Thực nghiệm trên 75 JD công khai của ba doanh nghiệp tạo ra 589 đoạn dữ liệu. Với tập đánh giá tự truy hồi gồm 15 tiêu đề JD, phương pháp hybrid đạt Recall@5 bằng 1,000, MRR bằng 0,9556 và tỷ lệ JD duy nhất trong kết quả bằng 1,000. Năm ca kiểm thử tự động đều đạt và frontend được biên dịch thành công mà không còn cảnh báo bảo mật từ npm audit. Kết quả cho thấy kiến trúc đáp ứng được mục tiêu của Project 1, đồng thời vẫn còn dư địa đánh giá trên bộ truy vấn do chuyên gia nhân sự gán nhãn và triển khai giám sát ở quy mô lớn.

**Từ khóa:** RAG, mô tả công việc, tuyển dụng, semantic search, hybrid retrieval, Gemini, Milvus.

## ABSTRACT

Recruitment teams need consistent job descriptions, explicit evaluation criteria, and role-specific interview questions. When reference data is fragmented and authoring depends heavily on individual experience, outputs become difficult to standardize, update, and trace. This report presents SmartHire Composer, a web-based system for creating, editing, versioning, and exporting job descriptions and generating interview questions. Retrieval-Augmented Generation (RAG) grounds generation in real public job postings.

The system consists of a React/Vite frontend, a FastAPI service, PostgreSQL for transactional data and full-text retrieval, Milvus for vector retrieval, and Gemini models for text generation and embeddings. Public ATS postings are normalized with provenance metadata, segmented into overlapping chunks, and embedded into 768-dimensional vectors. At query time, dense cosine retrieval and PostgreSQL lexical retrieval are fused using Reciprocal Rank Fusion, followed by diversity-aware selection. JWT authentication, role-based access control, version history, PDF/DOCX export, and graceful LLM fallback are also provided.

The experimental corpus contains 75 real job descriptions from three companies and 589 chunks. On a 15-query self-retrieval benchmark, hybrid retrieval achieved Recall@5 of 1.000, MRR of 0.9556, and a unique-document ratio of 1.000. All five automated tests passed, and the production frontend build completed with no npm audit vulnerabilities. These results demonstrate the feasibility of the proposed architecture while highlighting the need for expert-labeled queries, larger-scale evaluation, and production observability in future work.

**Keywords:** RAG, job description, recruitment, semantic search, hybrid retrieval, Gemini, Milvus.

# MỤC LỤC

|                                                 |            |
|-------------------------------------------------|------------|
| <b>Lời cam kết</b>                              | <b>i</b>   |
| <b>Lời cảm ơn</b>                               | <b>ii</b>  |
| <b>Tóm tắt</b>                                  | <b>iii</b> |
| <b>Abstract</b>                                 | <b>iv</b>  |
| <b>Danh mục công thức</b>                       | <b>ix</b>  |
| <b>Danh mục từ viết tắt</b>                     | <b>x</b>   |
| <b>1 GIỚI THIỆU ĐỀ TÀI</b>                      | <b>1</b>   |
| 1.1 Đặt vấn đề . . . . .                        | 1          |
| 1.2 Mục tiêu và phạm vi . . . . .               | 1          |
| 1.3 Định hướng giải pháp . . . . .              | 1          |
| 1.4 Đóng góp chính . . . . .                    | 2          |
| 1.5 Bố cục báo cáo . . . . .                    | 2          |
| <b>2 KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU</b>          | <b>3</b>   |
| 2.1 Khảo sát hiện trạng . . . . .               | 3          |
| 2.2 Tác nhân và chức năng . . . . .             | 3          |
| 2.3 Quy trình nghiệp vụ chính . . . . .         | 4          |
| 2.4 Yêu cầu phi chức năng . . . . .             | 5          |
| <b>3 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ</b>           | <b>6</b>   |
| 3.1 Mô hình ngôn ngữ lớn và embedding . . . . . | 6          |
| 3.2 Retrieval-Augmented Generation . . . . .    | 6          |
| 3.3 Truy hồi dense, lexical và hybrid . . . . . | 6          |
| 3.4 Chỉ mục HNSW . . . . .                      | 7          |
| 3.5 Xác thực và phân quyền . . . . .            | 7          |
| 3.6 Công nghệ triển khai . . . . .              | 7          |
| <b>4 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG</b>          | <b>8</b>   |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 4.1      | Kiến trúc tổng thể . . . . .                       | 8         |
| 4.2      | Thiết kế dữ liệu . . . . .                         | 8         |
| 4.3      | Thu thập và quản trị dữ liệu thật . . . . .        | 9         |
| 4.4      | Pipeline lập chỉ mục . . . . .                     | 9         |
| 4.5      | Thuật toán truy hồi hybrid . . . . .               | 10        |
| 4.6      | An toàn, độ tin cậy và khả năng quan sát . . . . . | 10        |
| 4.7      | Thiết kế giao diện và API . . . . .                | 11        |
| <b>5</b> | <b>TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ</b>          | <b>12</b> |
| 5.1      | Môi trường thực nghiệm . . . . .                   | 12        |
| 5.2      | Phương pháp đánh giá retrieval . . . . .           | 12        |
| 5.3      | Kết quả retrieval . . . . .                        | 13        |
| 5.4      | Kiểm thử phần mềm . . . . .                        | 13        |
| 5.5      | Thảo luận và đe dọa tính hợp lệ . . . . .          | 14        |
| <b>6</b> | <b>KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN</b>                | <b>15</b> |
| 6.1      | Kết luận . . . . .                                 | 15        |
| 6.2      | Hạn chế . . . . .                                  | 15        |
| 6.3      | Hướng phát triển . . . . .                         | 15        |
|          | <b>Tài liệu tham khảo</b>                          | <b>16</b> |
| <b>A</b> | <b>HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH</b>               | <b>17</b> |
| A.1      | Khởi tạo môi trường . . . . .                      | 17        |
| A.2      | Chạy ứng dụng . . . . .                            | 17        |
| A.3      | Thu thập, ETL và indexing . . . . .                | 17        |
| <b>B</b> | <b>DANH SÁCH API CHÍNH</b>                         | <b>18</b> |
| <b>C</b> | <b>CẤU TRÚC MÃ NGUỒN</b>                           | <b>19</b> |
| <b>D</b> | <b>KỊCH BẢN KIỂM THỬ TIÊU BIỂU</b>                 | <b>20</b> |

## DANH MỤC HÌNH VẼ

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 1.1 | Định hướng xử lý tổng quát của hệ thống . . . . . | 2  |
| 2.1 | Quy trình tạo và hoàn thiện một JD . . . . .      | 4  |
| 4.1 | Kiến trúc logic của SmartHire Composer . . . . .  | 8  |
| 4.2 | Pipeline lập chỉ mục RAG . . . . .                | 10 |
| 4.3 | Luồng truy hồi hybrid . . . . .                   | 10 |



## DANH MỤC BẢNG

|     |                                                              |    |
|-----|--------------------------------------------------------------|----|
| 2.1 | Yêu cầu chức năng của hệ thống . . . . .                     | 3  |
| 2.2 | Yêu cầu phi chức năng . . . . .                              | 5  |
| 3.1 | Các công nghệ chính . . . . .                                | 7  |
| 4.1 | Các bảng dữ liệu cốt lõi . . . . .                           | 9  |
| 5.1 | Thống kê corpus thực nghiệm . . . . .                        | 12 |
| 5.2 | Kết quả trên 15 truy vấn tự truy hồi, top- $k = 5$ . . . . . | 13 |
| 5.3 | Kết quả các nhóm kiểm thử . . . . .                          | 13 |

# DANH MỤC CÔNG THỨC

| Công thức | Nội dung                                           |
|-----------|----------------------------------------------------|
| 3.1       | Độ tương đồng cosine giữa query và document vector |
| 3.3       | Điểm Reciprocal Rank Fusion                        |
| 4.1       | Điểm lựa chọn có phạt trùng lặp Jaccard            |
| 5.1       | Mean Reciprocal Rank                               |

## DANH MỤC TỪ VIẾT TẮT

| Từ viết tắt | Giải nghĩa                                                       |
|-------------|------------------------------------------------------------------|
| API         | Application Programming Interface – giao diện lập trình ứng dụng |
| ATS         | Applicant Tracking System – hệ thống quản trị tuyển dụng         |
| CRUD        | Create, Read, Update, Delete                                     |
| DBMS        | Database Management System                                       |
| FTS         | Full-Text Search – tìm kiếm toàn văn                             |
| HNSW        | Hierarchical Navigable Small World                               |
| JD          | Job Description – mô tả công việc                                |
| JWT         | JSON Web Token                                                   |
| LLM         | Large Language Model – mô hình ngôn ngữ lớn                      |
| MMR         | Maximal Marginal Relevance                                       |
| MRR         | Mean Reciprocal Rank                                             |
| RAG         | Retrieval-Augmented Generation – sinh tăng cường bởi truy hồi    |
| RBAC        | Role-Based Access Control                                        |
| RRF         | Reciprocal Rank Fusion                                           |
| REST        | Representational State Transfer                                  |
| UI          | User Interface                                                   |

# CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

## 1.1 Đặt vấn đề

Mô tả công việc là một tài liệu trung tâm trong quy trình tuyển dụng: tài liệu này truyền đạt nhu cầu của đơn vị sử dụng lao động, định hướng nội dung đăng tuyển, làm đầu vào sàng lọc hồ sơ và là căn cứ xây dựng phỏng vấn. Tuy nhiên, việc soạn JD trong thực tế thường dựa trên các mẫu rời rạc, nội dung của tin cũ và kinh nghiệm của từng chuyên viên. Cách làm này dẫn tới ba nhóm vấn đề. Thứ nhất, cấu trúc và thuật ngữ giữa các JD thiếu đồng nhất. Thứ hai, người soạn khó tìm nhanh những yêu cầu tương tự trong kho dữ liệu lớn. Thứ ba, khi dùng mô hình ngôn ngữ để hỗ trợ viết, kết quả có thể chứa thông tin không có căn cứ nếu mô hình không được cung cấp ngữ cảnh phù hợp.

RAG là một hướng tiếp cận kết hợp hệ thống truy hồi thông tin với mô hình sinh. Thay vì yêu cầu LLM trả lời chỉ dựa trên tham số đã học, hệ thống truy hồi các đoạn dữ liệu liên quan rồi đưa chúng vào prompt. Trong bài toán này, dữ liệu truy hồi là các đoạn JD thật có thông tin nguồn. RAG không loại bỏ hoàn toàn sai sót của LLM, nhưng giúp tăng tính bám sát miền nghiệp vụ, cho phép truy vết nguồn và tách việc cập nhật tri thức khỏi quá trình huấn luyện lại mô hình [1].

## 1.2 Mục tiêu và phạm vi

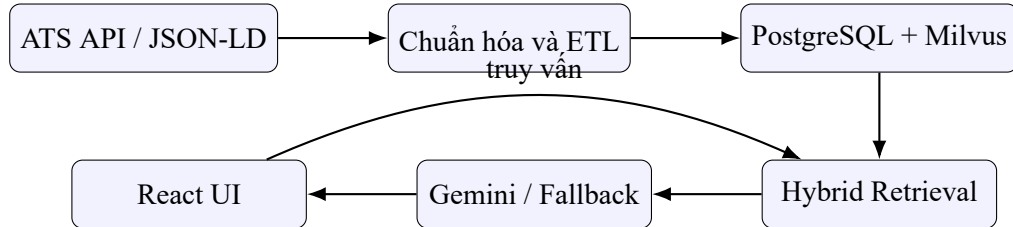
Mục tiêu của đề tài là xây dựng một hệ thống web có khả năng hỗ trợ toàn bộ vòng đời cơ bản của JD. Các mục tiêu cụ thể gồm: (i) thu thập và chuẩn hóa dữ liệu JD công khai kèm provenance; (ii) tìm kiếm ngữ nghĩa và từ khóa trên kho JD; (iii) dùng kết quả truy hồi để hỗ trợ tạo hoặc cải thiện JD; (iv) sinh câu hỏi phỏng vấn theo vị trí, cấp độ và nhóm năng lực; (v) quản lý người dùng, quyền truy cập và lịch sử phiên bản; và (vi) đánh giá định lượng chất lượng truy hồi.

Phạm vi Project 1 tập trung vào nguyên mẫu vận hành trên một máy phát triển. Corpus thực nghiệm gồm 75 JD công khai từ Cloudflare, Datadog và Figma. Hệ thống chưa xử lý hồ sơ ứng viên thật, không tự động đưa ra quyết định tuyển dụng và không thu thập dữ liệu cá nhân từ biểu mẫu ứng tuyển. Các API công khai của ATS được ưu tiên; các nguồn cấm bot hoặc yêu cầu đăng nhập không nằm trong phạm vi thu thập.

## 1.3 Định hướng giải pháp

Giải pháp được xây dựng theo kiến trúc nhiều tầng. React đảm nhiệm giao diện; FastAPI cung cấp REST API; PostgreSQL lưu dữ liệu nghiệp vụ, lịch sử và chỉ mục toàn

văn; Milvus lưu vector; Gemini cung cấp embedding và sinh văn bản. Pipeline truy hồi kết hợp dense retrieval với lexical retrieval, sử dụng RRF để hợp nhất thứ hạng và một bước chọn đa dạng để giảm trùng lặp. Các nguồn dữ liệu bên ngoài được coi là không đáng tin cậy trong prompt nhằm giảm rủi ro prompt injection.



**Hình 1.1.** Định hướng xử lý tổng quát của hệ thống

## 1.4 Đóng góp chính

Đề tài tạo ra một nguyên mẫu tích hợp được nhiều thành phần thường tách rời: thu thập JD thật có provenance, quản lý phiên bản, RAG hybrid, sinh nội dung, phân quyền và giao diện sử dụng. Điểm nhấn kỹ thuật là kho chunk đồng bộ giữa PostgreSQL và Milvus, manifest mô tả model/index, truy hồi hybrid RRF, giới hạn số chunk trên mỗi JD và benchmark có thể chạy lại. Nhờ đó, hệ thống không chỉ trình diễn LLM mà còn hình thành một quy trình dữ liệu có khả năng kiểm tra và tái lập.

## 1.5 Bố cục báo cáo

Chương 2 khảo sát hiện trạng và phân tích yêu cầu. Chương 3 trình bày cơ sở lý thuyết cùng các công nghệ sử dụng. Chương 4 mô tả kiến trúc, thiết kế dữ liệu, pipeline RAG và quá trình hiện thực. Chương 5 trình bày môi trường, phương pháp thực nghiệm, kết quả kiểm thử và đánh giá. Chương 6 tổng kết kết quả, chỉ ra hạn chế và đề xuất hướng phát triển.

## CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

### 2.1 Khảo sát hiện trạng

Các công cụ soạn thảo phổ biến có thể lưu mẫu JD nhưng chưa giải quyết tốt truy hồi ngữ nghĩa. Công cụ LLM độc lập hỗ trợ viết nhanh nhưng thường thiếu dữ liệu nội bộ và provenance. Hệ thống tìm kiếm từ khóa truyền thống chính xác khi truy vấn trùng thuật ngữ, song khó nhận biết các cách diễn đạt gần nghĩa. Vì vậy, đề tài lựa chọn cách kết hợp: cơ sở dữ liệu quản lý sự thật nghiệp vụ; lexical retrieval bảo toàn từ khóa hiếm; dense retrieval tìm tương đồng ngữ nghĩa; LLM chỉ tham gia sau khi đã có ngữ cảnh.

### 2.2 Tác nhân và chức năng

Hệ thống có bốn vai trò. *Admin* quản trị và sử dụng toàn bộ chức năng; *Recruiter* tạo JD, sinh câu hỏi, cập nhật phiên bản và xuất tài liệu; *Manager* xem lịch sử, role và truy hồi; *Viewer* là vai trò dự phòng cho chức năng chỉ đọc trong tương lai. Bảng 2.1 mô tả yêu cầu chức năng chính.

**Bảng 2.1.** Yêu cầu chức năng của hệ thống

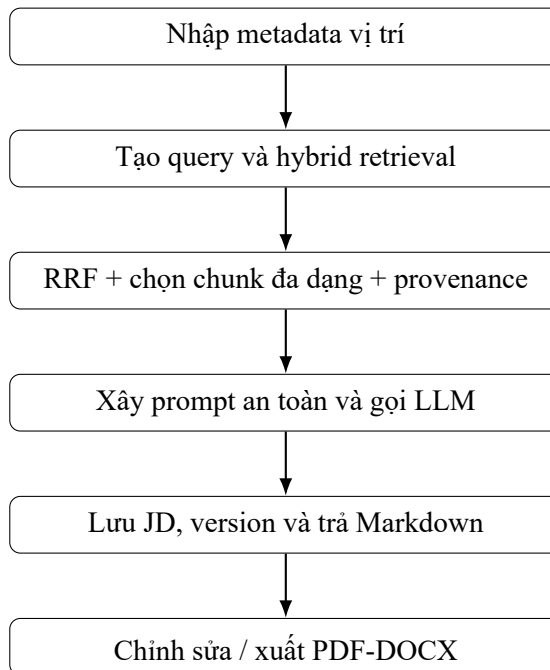
| Mã  | Chức năng         | Mô tả                                                                                   |
|-----|-------------------|-----------------------------------------------------------------------------------------|
| F01 | Xác thực          | Đăng nhập bằng username/password, nhận JWT và lấy hồ sơ hiện tại.                       |
| F02 | Soạn JD           | Nhập title, department, level, job family và ngôn ngữ để tạo bản nháp.                  |
| F03 | Quản lý phiên bản | Lưu mỗi lần chỉnh sửa, hiển thị lịch sử, người sửa và tóm tắt thay đổi.                 |
| F04 | Gợi ý/cải thiện   | Cải thiện toàn văn hoặc đề xuất nội dung cho một mục của JD.                            |
| F05 | Sinh phỏng vấn    | Sinh technical, behavioral và situational questions kèm competency, difficulty, rubric. |
| F06 | Truy hồi          | Tìm theo dense, lexical hoặc hybrid; lọc công ty/phòng ban; trả snippet và URL nguồn.   |
| F07 | Xuất tài liệu     | Xuất JD hiện hành thành PDF hoặc DOCX.                                                  |
| F08 | Thu thập dữ liệu  | Lấy JD từ Greenhouse, Lever hoặc JSON-LD và lưu provenance.                             |
| F09 | Lập chỉ mục       | Chia đoạn, embedding, ghi Milvus, catalog PostgreSQL và manifest.                       |

F10 Giám sát

Cung cấp liveness/readiness và request ID cho từng yêu cầu.

## 2.3 Quy trình nghiệp vụ chính

Quy trình tạo JD bắt đầu khi recruiter đăng nhập và nhập metadata của vị trí. Backend xây truy vấn từ title, level, department và job family. Hybrid retriever lấy các chunk liên quan; mỗi chunk được gắn công ty, tiêu đề và URL. Orchestrator đóng gói dữ liệu trong vùng UNTRUSTED\_SOURCE, yêu cầu LLM không thực thi chỉ dẫn từ nguồn, sau đó sinh bản nháp. JD và version đầu tiên được lưu trong cùng luồng nghiệp vụ. Người dùng có thể chỉnh sửa, tạo version mới và xuất tài liệu.



**Hình 2.1.** Quy trình tạo và hoàn thiện một JD

## 2.4 Yêu cầu phi chức năng

**Bảng 2.2.** Yêu cầu phi chức năng

| Nhóm      | Yêu cầu                                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------|
| Bảo mật   | Mật khẩu băm bcrypt; API bảo vệ bằng JWT/RBAC; secret không lưu trong mã nguồn; CORS theo allow-list. |
| Tin cậy   | Backend vẫn khởi động khi Milvus tạm lỗi; hybrid tự hạ xuống lexical nếu embedding không sẵn sàng.    |
| Truy vết  | Dữ liệu JD có source ID, URL, company, thời điểm lấy và SHA-256; request có X-Request-ID.             |
| Hiệu năng | HNSW cho vector; GIN cho FTS; cache tối đa 512 query embedding; giới hạn candidate và top-k.          |
| Duy trì   | Module hóa theo API/service/CRUD/schema; migration có thứ tự; kiểm thử tự động và script bootstrap.   |
| Khả dụng  | UI responsive; hỗ trợ tiếng Việt/Anh; Swagger mô tả API; xuất PDF/-DOCX.                              |



## CHƯƠNG 3. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

### 3.1 Mô hình ngôn ngữ lớn và embedding

LLM biểu diễn xác suất của chuỗi token và sinh token tiếp theo dựa trên ngữ cảnh. Trong hệ thống, LLM được dùng cho tác vụ sinh JD, cải thiện nội dung và sinh câu hỏi; nó không phải nguồn dữ liệu nghiệp vụ. Embedding ánh xạ một đoạn văn bản  $x$  sang vector  $\mathbf{e}(x) \in \mathbb{R}^{768}$ . Các văn bản gần nghĩa kỳ vọng có vector gần nhau. Độ tương đồng cosine được tính bởi

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2}. \quad (3.1)$$

Vector trong đề tài được chuẩn hóa L2 trước khi lưu. Gemini sử dụng hai chế độ embedding riêng cho tài liệu và truy vấn nhằm phân biệt vai trò biểu diễn của hai loại đầu vào [6].

### 3.2 Retrieval-Augmented Generation

RAG gồm hai giai đoạn chính: truy hồi bằng query và sinh có điều kiện trên ngữ cảnh truy hồi. Nếu  $D$  là kho đoạn văn và  $R(q)$  là tập top- $k$ , đầu ra được biểu diễn khái quát bởi

$$y = G(q, R(q; D)), \quad (3.2)$$

trong đó  $G$  là mô hình sinh. Ưu điểm của RAG là cập nhật tri thức bằng cách cập nhật index, không cần huấn luyện lại LLM. Hạn chế là chất lượng cuối chịu ảnh hưởng trực tiếp từ chunking, embedding, ranking, độ sạch dữ liệu và thiết kế prompt.

### 3.3 Truy hồi dense, lexical và hybrid

Dense retrieval dùng vector và cosine nên nhận biết quan hệ ngữ nghĩa nhưng đôi khi bỏ qua mã kỹ năng, tên công nghệ hoặc cụm từ hiếm. Lexical retrieval trong PostgreSQL dùng `to_tsvector` và `websearch_to_tsquery`; title được gán trọng số A, heading/content trọng số B. Nhược điểm của lexical retrieval là phụ thuộc từ bề mặt. Hybrid retrieval kết hợp ưu điểm của hai phương pháp.

Đề tài dùng Reciprocal Rank Fusion [2]. Với tập các bộ xếp hạng  $\mathcal{R}$ , điểm của tài liệu  $d$  là

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + r(d)}, \quad (3.3)$$

trong đó  $r(d)$  là thứ hạng và  $k = 60$  làm giảm ảnh hưởng của chênh lệch thứ hạng nhỏ. Sau RRF, hệ thống giới hạn số chunk mỗi JD và phạt Jaccard similarity với kết quả đã chọn, theo

tinh thần MMR [3].

### 3.4 Chỉ mục HNSW

Milvus sử dụng HNSW, một đồ thị nhiều tầng cho approximate nearest neighbor search [4]. Tham số  $M = 32$  điều khiển số liên kết; `efConstruction=200` điều khiển độ rộng khi xây chỉ mục; `ef` khi truy vấn được chọn theo số candidate và không nhỏ hơn 128. Cấu hình này ưu tiên recall cho corpus cỡ nhỏ và trung bình.

### 3.5 Xác thực và phân quyền

Sau khi xác minh mật khẩu, backend tạo JWT chứa `sub`, danh sách role và thời hạn. `Dependency require_roles` kiểm tra giao của role người dùng với role được phép. Cơ chế này tách xác thực khỏi logic endpoint, đồng thời cho phép Swagger sử dụng OAuth2 password flow. JWT không mã hóa payload; do đó hệ thống chỉ lưu định danh và quyền, không đưa dữ liệu nhạy cảm vào token [5].

### 3.6 Công nghệ triển khai

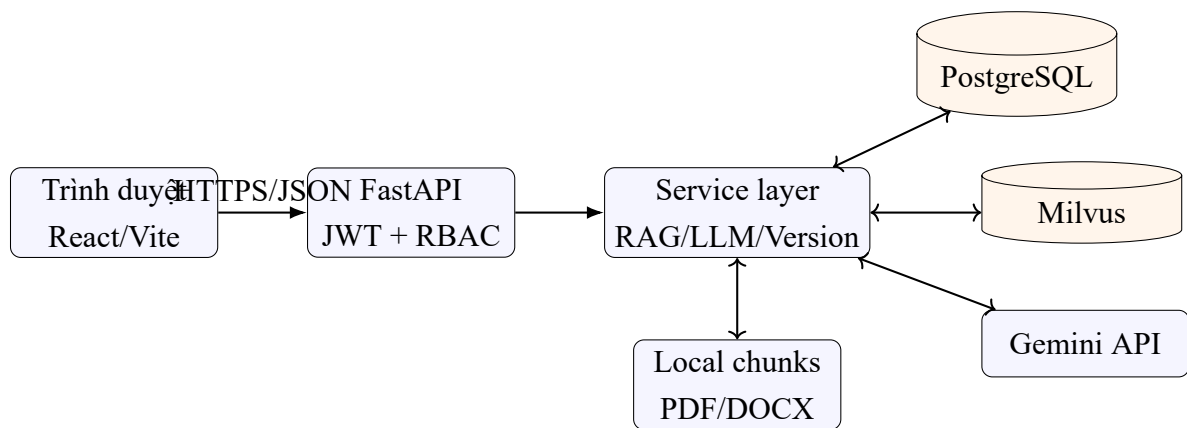
**Bảng 3.1.** Các công nghệ chính

| Thành phần | Công nghệ            | Vai trò                                              |
|------------|----------------------|------------------------------------------------------|
| Frontend   | React 18, Vite 8     | SPA, routing, editor, preview và gọi API.            |
| Backend    | Python 3.12, FastAPI | REST API, validation, dependency injection, Swagger. |
| Nghiệp vụ  | PostgreSQL 15        | JD, version, user, provenance, chunk catalog và FTS. |
| Vector     | Milvus 2.3.4         | HNSW/COSINE trên vector 768 chiều.                   |
| AI         | Gemini               | Sinh nội dung và <code>gemini-embedding-001</code> . |
| Hạ tầng    | Docker Compose       | PostgreSQL, Milvus, etcd và MinIO nội bộ.            |
| Kiểm thử   | pytest, npm          | Unit/integration test, build và dependency audit.    |
|            | build/audit          |                                                      |

## CHƯƠNG 4. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

### 4.1 Kiến trúc tổng thể

Hệ thống áp dụng kiến trúc phân lớp. Frontend không truy cập trực tiếp cơ sở dữ liệu; mọi yêu cầu đi qua FastAPI và dependency xác thực. Service layer điều phối LLM, retrieval, versioning và export. CRUD/ORM đóng gói truy cập PostgreSQL. Milvus được kết nối lười để sự cố vector store không ngăn backend khởi động. Hình 4.1 mô tả các thành phần triển khai.



Hình 4.1. Kiến trúc logic của SmartHire Composer

### 4.2 Thiết kế dữ liệu

PostgreSQL chứa hai nhóm dữ liệu. Nhóm nghiệp vụ gồm `users`, `roles`, `user_roles`, `job_descriptions`, `jd_versions`, `taxonomy` và `candidate schema` dự phòng. Nhóm RAG gồm `jd_chunks` và `rag_index_metadata`. `jd_chunks` lưu cùng `chunk_id` với Milvus, nội dung, hash, heading và ước lượng token; nhờ đó lexical retrieval và dense retrieval có thể hợp nhất bằng khóa ổn định. Manifest lưu provider, model, dimension, phiên bản chunker và số lượng chunk. Trước dense retrieval, cấu hình runtime được đối chiếu manifest để ngăn trộn hai không gian vector.

**Bảng 4.1.** Các bảng dữ liệu cốt lõi

| Bảng                   | Nội dung                                                                |
|------------------------|-------------------------------------------------------------------------|
| job_descriptions       | JD hiện hành, metadata tuyển dụng, provenance, hash nguồn và version.   |
| jd_versions            | Snapshot Markdown theo version, người sửa, thời gian và change summary. |
| jd_chunks              | Chunk text, heading, hash, đường dẫn, kích thước và liên kết JD.        |
| rag_index_metadata     | Model embedding, dimension, chunker version, document/chunk count.      |
| users/roles/user_roles | Đặc quyền và quan hệ nhiều-nhiều phục vụ RBAC.                          |
| job_families/tags      | Taxonomy nhóm nghề và nhãn nội dung.                                    |

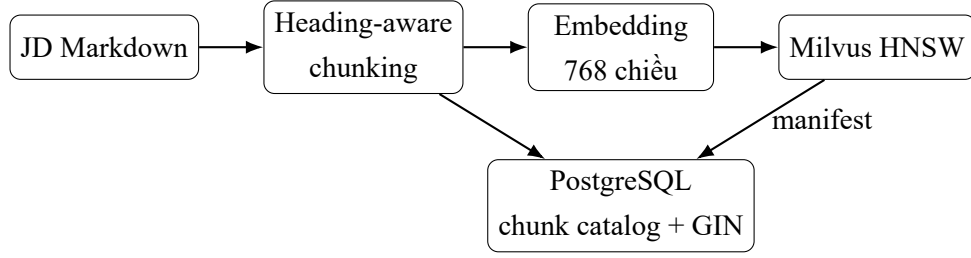
### 4.3 Thu thập và quản trị dữ liệu thật

Crawler hỗ trợ Greenhouse Job Board API, Lever Postings API và JSON-LD JobPosting. Adapter chuyển mỗi nguồn về cấu trúc thống nhất gồm external ID, company, title, description, URL, department, location, employment type và thời gian đăng. Nội dung được lưu thành Markdown kèm front matter. SHA-256 giúp nhận biết thay đổi; ETL upsert theo cặp `source_name`, `source_external_id`. Khi hash không đổi, hệ thống chỉ cập nhật thời điểm quan sát, không tạo version rác.

Pipeline tuân theo nguyên tắc chỉ lấy dữ liệu công khai qua API/URL được phép, có User-Agent, delay và attribution. Hệ thống không truy cập private API, không vượt CAPTCHA và không thu thập thông tin ứng viên. 109 JD synthetic cũ được tách khỏi corpus production; cơ sở dữ liệu thực nghiệm chỉ nhập 75 JD thật.

### 4.4 Pipeline lập chỉ mục

Chunker ưu tiên ranh giới heading, đoạn và câu; block dài được hard-split tại biên từ với overlap. Chunk có độ dài tối đa 1.200 ký tự, trung bình 1.007 ký tự. Embedding được tạo trước khi thay đổi index, nên lỗi quota không để lại một index chỉ xây dựng một phần. Gemini được gọi theo batch 90 đoạn và nghỉ giữa các batch theo giới hạn free tier. Sau insert, collection được compact; catalog và manifest PostgreSQL chỉ cập nhật khi dense index đã thành công.



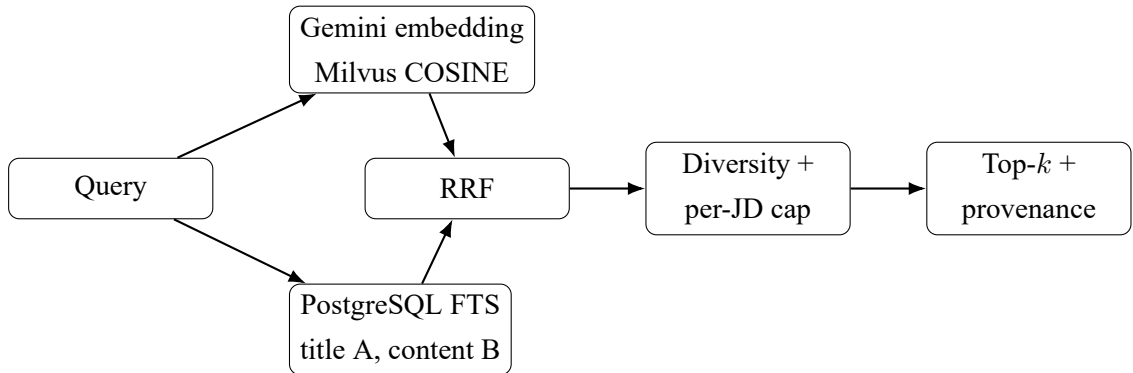
Hình 4.2. Pipeline lập chỉ mục RAG

## 4.5 Thuật toán truy hồi hybrid

Với một query, hệ thống lấy tối đa 40 dense candidates và 40 lexical candidates. Dense search có thể lọc tập JD hợp lệ theo company/department trước khi gửi biểu thức tới Milvus. Lexical search gán trọng số cao cho title và dùng GIN index cho content. Hai danh sách hợp nhất theo Công thức 3.3. Sau đó hệ thống chọn tuần tự ứng viên có điểm hiệu dụng lớn nhất:

$$s'(c) = s_{RRF}(c) - 0.08 \max_{d \in S} J(T(c), T(d)), \quad (4.1)$$

trong đó  $J$  là Jaccard trên token và  $S$  là tập đã chọn. Mặc định không quá hai chunk từ một JD; khi tạo prompt chỉ lấy một chunk/JD để tăng độ phủ nguồn. Nếu embedding lỗi ở hybrid mode, endpoint tự chuyển sang lexical thay vì trả lỗi toàn bộ.



Hình 4.3. Luồng truy hồi hybrid

## 4.6 An toàn, độ tin cậy và khả năng quan sát

Nguồn JD bên ngoài được đặt trong thẻ UNTRUSTED\_SOURCE; prompt yêu cầu bỏ qua mọi chỉ dẫn trong nguồn và chỉ trích xuất dữ kiện nghề nghiệp. Hệ thống yêu cầu LLM tổng hợp thay vì sao chép đoạn dài và liệt kê URL đã dùng. Nếu LLM gặp 429/503, tạo JD trả về mẫu tối thiểu không bịa đặt; improve giữ nguyên bản gốc; interview generator chuyển sang bộ sinh offline xác định được bằng seed.

Middleware thêm X-Request-ID và Server-Timing. Endpoint /health/live

kiểm tra tiến trình; /health/ready kiểm tra PostgreSQL, Milvus, collection, số chunk catalog và số entity. Query embedding có cache LRU 512 phần tử. Những cơ chế này giúp phân biệt lỗi hạ tầng, lỗi model và lỗi nghiệp vụ.

## 4.7 Thiết kế giao diện và API

Frontend gồm Dashboard, Compose, Interview, Retrieve và Roles. Compose có form metadata, Markdown editor, preview đã sanitize bằng DOMPurify, lịch sử version và export. Retrieve cho phép chọn Hybrid, Dense hoặc Lexical và hiển thị score, phương pháp, công ty, title, URL, chunk path và snippet. API chính được tổng hợp ở Phụ lục B.

## CHƯƠNG 5. TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ

### 5.1 Môi trường thực nghiệm

Thử nghiệm được thực hiện trên Windows với Python 3.12, Node.js 25 và Docker Desktop. PostgreSQL 15, Milvus 2.3.4, etcd và MinIO chạy trong Docker Compose; backend và frontend chạy trực tiếp để hỗ trợ phát triển. Embedding model là `gemini-embedding-001`, chiều vector 768; collection dùng HNSW/COSINE.

**Bảng 5.1.** Thống kê corpus thực nghiệm

| Chỉ số                          | Giá trị      |
|---------------------------------|--------------|
| Số doanh nghiệp                 | 3            |
| Số JD thật                      | 75           |
| JD Cloudflare / Datadog / Figma | 25 / 25 / 25 |
| Số chunk                        | 589          |
| Độ dài chunk trung bình         | 1.007 ký tự  |
| Độ dài chunk tối đa             | 1.200 ký tự  |
| Chiều vector                    | 768          |

### 5.2 Phương pháp đánh giá retrieval

Script `evaluate_rag.py` lấy 15 JD thật đầu tiên, sử dụng title làm query và coi JD gốc là relevant document. Với danh sách top- $k$ ,  $\text{Recall}@k$  bằng tỷ lệ query mà JD gốc xuất hiện; MRR là trung bình nghịch đảo thứ hạng:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}. \quad (5.1)$$

Unique JD ratio đo số JD khác nhau chia cho số kết quả. Benchmark được chạy với tối đa một chunk/JD để đo độ phủ tài liệu.

### 5.3 Kết quả retrieval

**Bảng 5.2.** Kết quả trên 15 truy vấn tự truy hồi,  $\text{top-}k = 5$

| Phương pháp                      | Recall@5 | MRR    | Unique JD ratio |
|----------------------------------|----------|--------|-----------------|
| Lexical (sau chuẩn hóa dấu gạch) | 1,0000   | 1,0000 | 1,0000          |
| Dense                            | 1,0000   | 0,9556 | 1,0000          |
| Hybrid RRF                       | 1,0000   | 0,9556 | 1,0000          |

Trước bước giới hạn/đa dạng hóa, top-10 của ba truy vấn quan sát chỉ chứa 8–9 JD duy nhất; một JD có thể chiếm ba vị trí. Sau per-JD cap, các benchmark top-5 đạt unique ratio 1,0. Lexical ban đầu bỏ lỡ title có dấu gạch vì `websearch_to_tsquery` diễn giải dấu “-” như toán tử phủ định; chuẩn hóa dấu phân tách đưa Recall@5 và MRR lexical từ 0,8 lên 1,0. Kết quả này minh họa giá trị của phân tích lỗi thay vì chỉ điều chỉnh model.

Dense và hybrid có MRR thấp hơn lexical trên self-title benchmark vì title là tín hiệu từ khóa gần như hoàn hảo. Điều này không chứng minh lexical tốt hơn với truy vấn tự nhiên; dense/hybrid được kỳ vọng có lợi khi người dùng diễn đạt theo nhiệm vụ hoặc kỹ năng thay vì lặp title. Cần tập đánh giá do chuyên gia gán nhãn để kiểm chứng giả thuyết đó.

### 5.4 Kiểm thử phần mềm

**Bảng 5.3.** Kết quả các nhóm kiểm thử

| Nhóm              | Nội dung                                                        | Kết quả    |
|-------------------|-----------------------------------------------------------------|------------|
| Chunking          | Input rộng, block dài, giới hạn kích thước và bảo toàn nội dung | Đạt        |
| Milvus schema     | Collection và các field bắt buộc                                | Đạt        |
| Lexical retrieval | Tính liên quan và tính đa dạng                                  | Đạt        |
| Metadata filter   | Lọc kết quả theo company                                        | Đạt        |
| Backend E2E       | Login, /auth/me, version, export, retrieval, interview fallback | Đạt        |
| Frontend          | Vite production build                                           | Đạt        |
| Dependency audit  | npm audit                                                       | 0 lỗi hồng |

Tại lần đo cuối, pytest báo 5 passed. Vite 8 biên dịch 383 module thành bundle JavaScript khoảng 450 KB trước gzip và npm audit không phát hiện lỗi hồng đã biết. Post-



greSQL FTS dùng Bitmap Index Scan trên GIN; truy vấn mẫu “sales account executive” hoàn thành khoảng 0,195 ms trong môi trường local.

## 5.5 Thảo luận và đe dọa tính hợp lệ

Tập dữ liệu có provenance thật nhưng còn nhỏ và chỉ gồm ba doanh nghiệp công nghệ, vì vậy phân phối chức danh, ngôn ngữ và địa lý chưa đại diện thị trường lao động. Benchmark tự truy hồi bằng title thiên vị lexical và chưa đo relevance theo nhu cầu HR. Recall cao không phản ánh trực tiếp chất lượng nội dung sinh. LLM là dịch vụ ngoài, chịu quota và biến động 503; fallback bảo toàn chức năng nhưng chất lượng thấp hơn. Ngoài ra, báo cáo chưa có đánh giá người dùng, latency dưới tải đồng thời, chi phí token theo tháng và kiểm thử bảo mật chuyên sâu.

Để giảm các đe dọa này, hệ thống lưu manifest/timestamp, cung cấp script benchmark chạy lại, tách corpus synthetic, giữ URL nguồn và công bố rõ cấu hình. Các số liệu trong báo cáo vì vậy nên được hiểu là kết quả của một nguyên mẫu có thể tái lập, không phải kết luận tổng quát cho mọi miền tuyển dụng.

## CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 6.1 Kết luận

Đề tài đã xây dựng được SmartHire Composer với các chức năng chính: xác thực và RBAC; tạo, chỉnh sửa, lưu version và xuất JD; sinh câu hỏi phỏng vấn; thu thập dữ liệu thật có provenance; lập chỉ mục vector; và truy hồi dense/lexical/hybrid. Về kỹ thuật, hệ thống kết hợp PostgreSQL và Milvus bằng khóa chunk ổn định, kiểm soát tương thích embedding bằng manifest, xử lý đa dạng kết quả và phòng vệ prompt injection ở mức ứng dụng. Corpus gồm 75 JD thật và 589 chunk; benchmark 15 query đạt Recall@5 bằng 1,0 cho cả ba chế độ sau tối ưu.

Kết quả quan trọng của Project 1 không chỉ là một giao diện gọi LLM, mà là quy trình dữ liệu từ nguồn đến retrieval và generation có thể truy vết, kiểm thử và đánh giá. Kiến trúc module hóa cho phép thay embedding model, thêm nguồn ATS hoặc mở rộng reranker mà không thay đổi toàn bộ hệ thống.

### 6.2 Hạn chế

Hệ thống chưa có tập relevance do chuyên gia nhân sự gán nhãn; chưa đo chất lượng JD sinh bằng rubric; chưa có scheduler production cho crawler và incremental vector update hoàn chỉnh; chưa đánh dấu tự động tin đóng; chưa có multi-tenant isolation; và chưa triển khai secret manager, HTTPS, backup, tracing phân tán hay load test quy mô lớn. Local lexical fallback chỉ dựa trên hashing, không tương đương semantic embedding.

### 6.3 Hướng phát triển

Trong giai đoạn tiếp theo, đề tài có thể mở rộng theo bốn hướng. Thứ nhất, xây bộ đánh giá gồm truy vấn tự nhiên, relevance judgment và rubric chất lượng sinh; tích hợp RAGAS hoặc đánh giá pairwise có kiểm soát. Thứ hai, thêm cross-encoder reranker, query expansion và learning-to-rank, đồng thời A/B test trọng số RRF. Thứ ba, hoàn thiện pipeline dữ liệu với scheduler, incremental indexing theo hash, phát hiện tin đóng và retention policy. Thứ tư, nâng cấp production bằng HTTPS, secret manager, OpenTelemetry, Redis cache, worker queue, PostgreSQL backup và kiểm thử tải. Với miền tuyển dụng, cần bổ sung cơ chế kiểm tra thiên lệch, ngôn ngữ loại trừ và human-in-the-loop trước khi nội dung được công bố.

## TÀI LIỆU THAM KHẢO

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *Proceedings of SIGIR*, 2009.
- [3] J. Carbonell and J. Goldstein, “The Use of MMR, Diversity-Based Reranking for Re-ordering Documents and Producing Summaries,” *Proceedings of SIGIR*, 1998.
- [4] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, vol. 42, no. 4, 2020.
- [5] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [6] Google, “Gemini Embeddings,” Google AI for Developers. <https://ai.google.dev/gemini-api/docs/embeddings>, truy cập tháng 6/2026.
- [7] Milvus, “Milvus Documentation – Vector Search and HNSW Index,” <https://milvus.io/docs>, truy cập tháng 6/2026.
- [8] PostgreSQL Global Development Group, “PostgreSQL Full Text Search,” <https://www.postgresql.org/docs/15/textsearch.html>, truy cập tháng 6/2026.
- [9] FastAPI, “FastAPI Documentation,” <https://fastapi.tiangolo.com/>, truy cập tháng 6/2026.
- [10] Greenhouse Software, “Job Board API,” <https://developers.greenhouse.io/job-board.html>, truy cập tháng 6/2026.
- [11] Lever, “Lever Postings API,” <https://github.com/lever/postings-api>, truy cập tháng 6/2026.
- [12] Schema.org, “JobPosting,” <https://schema.org/JobPosting>, truy cập tháng 6/2026.

## CHƯƠNG A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH

### A.1 Khởi tạo môi trường

**Listing A.1.** Các lệnh khởi tạo trên PowerShell

```
1 py -3.12 -m venv .venv
2 .\.venv\Scripts\python.exe -m pip install -r requirements.txt
3 cd frontend
4 npm ci
5 cd ..
6 .\scripts\bootstrap.ps1
```

### A.2 Chạy ứng dụng

```
1 # Backend
2 $env:PYTHONUTF8="1"
3 .\.venv\Scripts\python.exe -m uvicorn src.main:app --host 0.0.0.0 --
  port 8000
4
5 # Frontend
6 cd frontend
7 npm run dev -- --host 0.0.0.0
```

### A.3 Thu thập, ETL và indexing

```
1 .\.venv\Scripts\python.exe scripts\crawl_jobs.py \
2 --source greenhouse --site cloudflare \
3 --company Cloudflare --limit 25
4
5 .\.venv\Scripts\python.exe etl\jd_etl.py \
6 --dir data\real_jd --author public-api-ingestion
7
8 .\.venv\Scripts\python.exe embeddings\jd_chunk_embed.py
```

## CHƯƠNG B. DANH SÁCH API CHÍNH

| Method | Endpoint                    | Mục đích                            |
|--------|-----------------------------|-------------------------------------|
| POST   | /auth/token                 | Đăng nhập và nhận JWT.              |
| GET    | /auth/me                    | Thông tin người dùng hiện tại.      |
| POST   | /v1/jd/generate             | Tạo JD, có thể dùng RAG.            |
| PUT    | /v1/jd/update               | Lưu version mới.                    |
| GET    | /v1/jd/version-history/{id} | Đánh sách version.                  |
| POST   | /v1/jd/improve              | Cải thiện JD.                       |
| POST   | /v1/jd/suggest              | Gợi ý cho một section.              |
| GET    | /v1/jd/export/{id}          | Xuất PDF/DOCX.                      |
| POST   | /v1/interview/generate      | Sinh câu hỏi phỏng vấn.             |
| POST   | /v1/retrieve                | Hybrid/dense/lexical retrieval.     |
| POST   | /v1/retrieve/similar        | Dense retrieval tương thích API cũ. |
| GET    | /health/live                | Liveness.                           |
| GET    | /health/ready               | Readiness toàn bộ storage/index.    |

## CHƯƠNG C. CẤU TRÚC MÃ NGUỒN

**Listing C.1.** Cấu trúc thư mục chính

```
1 Project1/
2 |-- src/ # FastAPI, service, ORM, schema
3 |-- frontend/ # React/Vite
4 |-- embeddings/ # chunking, embedding, indexing
5 |-- etl/ # Markdown -> PostgreSQL
6 |-- scripts/ # crawler, benchmark, bootstrap
7 |-- infra/migrations/ # PostgreSQL migrations
8 |-- data/real_jd/ # JD that co provenance
9 |-- data/chunks/ # chunk Markdown local
10 |-- tests/, test/ # automated tests
11 `-- report/ # bao cao LaTeX
```

## CHƯƠNG D. KỊCH BẢN KIỂM THỬ TIÊU BIỂU

| Mã  | Tiền điều kiện / dữ liệu      | Thao tác                        | Kết quả mong đợi                       |
|-----|-------------------------------|---------------------------------|----------------------------------------|
| T01 | User hr_admin đã seed         | POST /auth/token                | JWT Bearer hợp lệ.                     |
| T02 | Có JWT recruiter              | Tạo JD mới                      | JD và version 1 được lưu.              |
| T03 | Có JD                         | PUT update hai lần              | Version tăng tuần tự, history mới–cũ.  |
| T04 | Milvus và catalog đồng bộ     | Query “sales account executive” | Top-5 liên quan, không lặp JD.         |
| T05 | Gemini embedding lỗi tạm thời | Hybrid retrieval                | Tự chuyển lexical, không làm hỏng API. |
| T06 | Company=Figma                 | Truy hồi có filter              | Tất cả kết quả thuộc Figma.            |
| T07 | Nội dung chứa ký tự đặc biệt  | Xuất PDF/DOCX                   | HTTP 200, file mở được.                |
| T08 | Milvus thiếu collection       | GET readiness                   | HTTP 503 với check index=false.        |